

Research

Voice-activated home automation system for IoT edge devices using TinyML

Timothy Malche¹ · Sandeep Budhani² · Pramod Kumar Soni¹ · Govind Murari Upadhyay¹

Received: 12 February 2025 / Accepted: 22 May 2025

Published online: 06 June 2025

© The Author(s) 2025 [OPEN](#)

Abstract

Home automation systems are popular because they enhance the quality of life and the way users interact with the environment. Deploying complex machine learning models on Internet of Things (IoT) devices with limited resources is still difficult. This study proposes a home automation system based on a TinyML (Tiny Machine Learning) model to recognize specific spoken keywords. The developed model runs effectively on IoT devices which usually have limited resources. Using TinyML, the limitations of memory size, processing power and latency associated with IoT devices are addressed. The objective of this research is to train a keyword-spotting model for devices with low computation and memory. The trained TinyML model can recognize specific voice commands associated with home automation tasks, such as controlling lights, thermostats, and other appliances. To test our approach, we ran experiments in real-world settings and on edge IoT devices with limited resources. The results show that our keyword spotting model is both highly accurate and efficient and uses minimum computational resources. This research helps in the advancement of TinyML applications in home automation and broadens the potential for voice interaction in constrained environments. The keyword spotting model in the proposed system is built using Deep Convolutional Neural Network (DCNN). Different data pre-processing techniques are also applied to refine the dataset. The trained model is then converted to be deployed on the low resource devices without compromising the model's efficiency. The model attains an 96.67% test accuracy. The model is quantized for devices with limited resources. It operates with an 11 ms latency, using 19.8 K of RAM and 55.0 K of flash for recognizing and classifying users' voice commands in real-time. This demonstrates how TinyML can create efficient and user-friendly smart home solutions. The main contribution of the work presented in this paper is that the designed model can be deployed on a wide range of IoT devices. Since the model is trained on voice instructions which limits the model's robustness. In future work, this limitation can be eliminated by integrating multilingual instructions.

Article Highlights

- Demonstrates how voice control can run on tiny, low-power chips for smart home use.
- Shows that accurate voice recognition is possible without cloud or high-end hardware.
- Opens the door to smarter, easier home automation using speech on low-cost devices.

Timothy Malche, Sandeep Budhani, Pramod Kumar Soni and Govind Murari Upadhyay contributed equally to this work.

✉ Govind Murari Upadhyay, govind.upadhyay@jaipur.manipal.edu; Timothy Malche, timothy.malche@jaipur.manipal.edu; Sandeep Budhani, sandeebudhani13@gmail.com; Pramod Kumar Soni, pramod.soni@jaipur.manipal.edu | ¹Department of Computer Applications, Manipal University Jaipur, Jaipur, Rajasthan 303007, India. ²Department of CS & E, Graphic Era Hill University Bhimtal Campus, Bhimtal, Uttarakhand 263136, India.



Keywords Internet of Things · TinyML · Machine learning · Keyword spotting · Home automation · Good health and well-being

1 Introduction

The need for voice-activated smart home automation solutions is rising as IoT devices are increasingly being adopted in contemporary households. Homeowners benefit from the simplicity and usability of voice-controlled home automation since it eliminates the need for physical interfaces and lets users naturally engage with their devices. However, because of their confined processing power and memory, IoT devices with limited resources find it difficult to develop complicated machine-learning models for voice recognition [1–3].

By offering a compact keyword-spotting model appropriate for IoT devices with limited resources, this research seeks to alleviate this problem. Identifying certain words or phrases from audio inputs is known as “keyword spotting,” which enables effective and precise voice command recognition on low-power systems. The suggested model is enhanced to function effectively on IoT devices with constrained resources using TinyML approaches.

As illustrated in the above Fig. 1, the key components of a typical Keyword Spotting (KWS) system are a feature extractor and a classifier which is based on a neural network. Initially, the input voice signal, represented by L , is separated into frames of shorter length, denoted by I . These frames overlap to some amount, as determined by the s stride. T frames are obtained as a result of this framing technique and can be represented using mathematical formula given below [4].

$$T = \frac{L - I}{s} + 1 \quad (1)$$

Mel frequency spectral coefficients (MFCC) is a feature extraction technique popular in speech processing. MFCC translates speech input from the time domain to frequency domain spectral coefficients. This conversion makes it easier to reduce the size of the input signal. The resulting matrix of voice characteristics is fed into a classifier module which computes the probabilities corresponding to the potential output classes. This research proposes a novel approach to designing a keyword-detecting system for home automation. This system is built on Convolutional Neural Networks (CNN), a deep learning technology known for its effectiveness in pattern recognition applications. The MFCC approach, derived from classic speech processing techniques, is used in the feature extraction procedure. The system uses MFCC to convert the raw audio input into a compact collection of spectral coefficients.

The suggested design includes an additional functionality called SpecAugment. By adding controlled variations into spectrograms, this augmentation strategy improves the training dataset and strengthens the model's capacity to generalize well across varied contexts. The CNN design has been quantized to run seamlessly on low-resource micro-controllers, making it a good option for use in devices with limited computing capability. The inference phase, which involves the model processing new data and making predictions, is carried out locally. This design decision solves the issue of latency, which is a typical concern in real-time applications. In particular, this local inference reduces the dependency on cloud-based processing, distinguishing this system from competitors like Amazon Alexa and Google Dot Echo. This research presents a unique keyword detection system for home automation that uses CNN architecture, MFCC feature extraction, SpecAugment for data augmentation, and local inference.

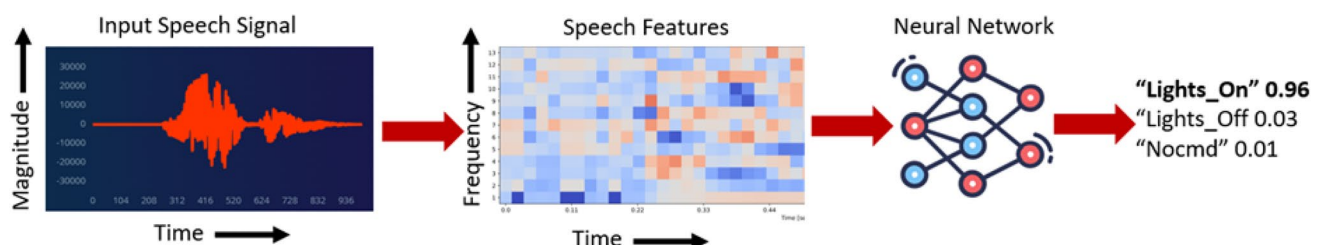


Fig. 1 Keyword spotting pipeline

Conventional cloud-based KWS solutions, such as those used in Amazon Alexa or Google Assistant, rely on sending voice data to remote servers for processing [5]. While these models benefit from high computational power and large-scale neural networks, they suffer from significant latency due to network transmission delays which makes them less suitable for real-time, low-latency applications in smart home automation [6]. Cloud-based KWS solutions also raise privacy concerns, as sensitive voice data is transmitted over the internet and stored remotely [7]. In contrast, TinyML-based KWS solutions use lightweight Deep Convolutional Neural Networks (DCNNs) that are quantized and optimized to run efficiently on microcontrollers and low-power IoT devices, offering real-time voice recognition with low latency. Unlike cloud-based solutions, which require constant internet connectivity, TinyML-based models process voice commands locally, ensuring faster responses, reduced power consumption, and enhanced user privacy. This shift towards on-device inference makes TinyML the most viable option for energy-efficient, real-time keyword spotting in resource-constrained smart home applications.

Edge-based Keyword Spotting (KWS) systems also face several limitations such as, these systems often operate on low-power devices with limited memory and processing capabilities [8, 9]. Therefore the ML model should be designed in keeping such limitation in mind [10]. Similarly, the always-on nature of KWS requires immediate response times, which can be difficult to achieve with resource-constrained hardware if the design of ML model is not efficient. We have addressed such issues in our research.

The main objective of this study is to develop a lightweight keyword-spotting model capable of operating efficiently on resource-limited IoT devices for home automation applications. The proposed solution uses Tiny Machine Learning (TinyML) techniques to enable efficient and accurate voice command recognition while minimizing memory usage, computational overhead, and latency. The following are the key contributions in this paper:

- Developed a DCNN-based TinyML model for recognizing keywords in home automation, designed to run on low-power IoT devices.
- Applied improved speech processing using MFCC and SpecAugment to diversify the training data and making the model robust.
- Enabled the model to work directly on IoT devices to reduce delays and ensure user privacy.
- Used 8-bit quantization to shrink the model size to reduce computational costs, achieve fast responses (11 ms latency) with minimal memory use.

The rest of the paper is organized as follows: Sect. 2 reviews existing keyword-spotting techniques and voice recognition approaches. Section 3 outlines the proposed system architecture and explains data collection, preprocessing, and feature extraction using MFCC and describes the DCNN model architecture. In Sect. 4, the training and validation of the proposed DCNN model is discussed. In Sect. 5, the quantization technique is used to optimize the model for IoT deployment along with experimental results with a comparison of unoptimized and quantized models. Section 6 summarizes the research findings and the benefits of using TinyML for keyword spotting.

2 Related work

Previous research in the field of home automation has explored various approaches to voice recognition, including cloud-based solutions and traditional machine-learning models. Cloud-based solutions rely on sending audio data to remote servers for processing. These cloud-based systems are efficient but they might face several issues such as latency, privacy concerns, real-time performance, and dependency on internet connectivity. Traditional machine learning models, such as deep neural networks, are computationally inefficient and require large amounts of memory, making them unsuitable for deployment on resource-constrained IoT devices. Recent advancements in TinyML have shown promising results in deploying lightweight machine learning models on IoT devices, enabling on-device processing and reducing reliance on cloud-based solutions. However, applying TinyML in the context of keyword spotting for home automation remains an underexplored area.

Chen et al. [11] have proposed an approach which is based on deep neural networks, while Majeed et al. [12] pointed out the concept of home automation for controlling and monitoring operations with the help of smart devices. Wang et al. [13] introduced a keyword-spotting system that uses deep neural networks and connectionist temporal classifiers. This approach is used for low-resource mobile devices. Guaman et al. [5] proposed a home automation device which is controlled through voice commands. Experiments are performed with the prototype of the device to control different

types of home appliances effectively. Choi et al. [14] proposed a temporal convolution using ResNet for mobile devices and achieved more than 385x speedup on Google Pixel.

Leroy et al. [15] investigated the application of federated learning to develop a wake-word detector using crowd-sourced speech data, focusing on resource-constrained environments. The research utilized the publicly available Hey Snips wake word dataset. The study introduced a modified version of the Federated Averaging Algorithm, incorporating a coordinate-wise averaging technique based on the Adam optimizer rather than the conventional global averaging approach. Bluche et al. [16] focused on an open vocabulary KWS method, which allows users to define custom keywords without the need to retain the whole model and described various designs for building fast and small-footprint systems for various kinds of user-defined keywords. This model could be run on tiny devices without training specific data to those keywords. Hard et al. [17] have designed a keyword spotting (KWS) model using federated learning (FedYogi) on both non-identical and identified data. This approach enabled authors to train a model with a lower false rejection rate in fewer training rounds. Additionally, by replacing memory-intensive MTR data augmentation with SpecAugment, the authors overcame resource constraints and reduced the false rejection rate by 56%. For these experimental studies, various optimization algorithms and hyper-parameter configurations were used on a large scale. Mittermaier et al. [18] addressed the significant impact of neural network size on power consumption, particularly noting that memory access operations consume more power than computations. To overcome this issue an energy-efficient keyword spotting (KWS) neural network architecture was developed. This innovative design combines feature extraction using SincConvs with GDSCnv layers, effectively reducing the total parameters. The proposed architecture gives better results on the test set from Google Speech Commands dataset, achieving an accuracy of 96.4% using only 62,000 parameters. This approach represents a significant advancement in balancing performance with energy efficiency for KWS systems, potentially enabling more widespread deployment of these technologies in power-constrained environments. Rybakov et al. [19] explored the latency and accuracy of the keyword spotting model in streaming and non-streaming modes on mobiles using the Tensorflow and Keras library to convert streaming mode to non-streaming mode automatically. Rafael Stahl et al. [20] focused on optimizing neural networks for memory-constrained edge devices and explored a distribution approach, often referred to as “Deeper Things,” which addresses both feature-intensive and weight-intensive convolutional layers. This method goes beyond simply distributing the computational load across devices. The key innovation of this work lies in its holistic approach to optimization. Rather than focusing on a single aspect of system performance, the approach jointly considered and optimized three critical factors: memory usage, computational demands, and communication requirements. Ivan et al. [21] created a novel KWS system called DKWS (Deep KWS) that uses speech sounds, acoustic models, and other methods to find specific keywords in spoken language. It also includes information about how to use it, what datasets to use, and how to measure its performance. Liu et al. [22] built RNN-T (RNN Transducer) based KWS that uses a constrained attention mechanism to focus on specific keywords. It is a multi-level system that achieves a 3.70% false rejection rate (FRR) at 1.0 false alarm (FA) per hour in a typical indoor environment dataset.

A novel training approach for keyword detection models based on the DNN-HMM architecture was proposed [23]. This method uses IOU-based sampling to tackle the issue of loss-metric mismatch. This technique has shown considerable improvements over traditional DNN-HMM training methods. When compared to CNN-based end-to-end models, this approach offers several advantages, such as better interpretability, accurate localization, and higher data efficiency. Immonen et al. [24] discussed resource optimization challenges of tiny ML and different methods like quantization, pruning and clustering, which can be used to eliminate these resource difficulties. Alajlan et al. [25] reviewed the revolution of tinyML and analyzed machine learning, which will be used in tinyML and presented details of datasets, its types and characteristics of used devices. He et al. [26] proposed a KWS based on a modified temporal efficient neural network (TENet) and a simplified MFCC algorithm using the accelerator architecture on FPGA and obtained an accuracy of 95.36%.

Ng et al. in [27] developed new feature interactive convolutional model with 100,000 parameters, to improve keyword spotting in noisy environments. This model uses an interactive unit instead of an attention module for more efficient information flow and obtained an accuracy of 98.2% top 1 on Google Speech Command V2-12 and used real-world scenarios. Ahmed et al. [28] designed the Ensemble for Keyword Spotting (EKOS) model, which was based on privacy and security concerns of two threats: first-experimental demonstration in which accidental activation results in up to a minute of speech recording being uploaded to the cloud server, and second adversarial activation, which exposes the integrity and availability services connected to VA. Alzoubi et al. [29] worked on energy consumption optimization and predicted 92% accuracy, which was greater than other previously used techniques.

The reviewed literature on keyword spotting (KWS) technology covers a broad spectrum, addressing aspects like deep neural network implementation, home automation integration, real-time KWS on mobile devices. However, potential research gaps include the need for integrated approaches, real-world validation, more robust models, standardized

benchmarking, and optimization for edge devices. Filling these gaps could enhance the practicality, scalability, and robustness of KWS systems, contributing to their effective deployment in diverse environments.

The Table 1 summarizes various keyword spotting approaches, detailing their methods, performance improvements, platforms, and datasets. Our proposed study, focus on detecting “Light On” and “Light Off” commands using MFCC features combined with a DNN. By quantizing the model, we ensure it meets the memory constraints of IoT devices, which is crucial for real-time, on-device processing. This approach reduces latency and enhances privacy since data processing occurs locally rather than in the cloud. Our system achieves a high accuracy of 96.67% on a custom dataset, striking an effective balance between performance and resource efficiency.

Recent advancements in voice-activated IoT solutions have largely relied on cloud-based architectures such as Amazon Alexa, Google Assistant, and Apple Siri. While these systems offer high recognition accuracy and integration with a broad ecosystem, they are heavily dependent on stable internet connectivity, which exhibit noticeable latency due to server-side processing and raises significant concerns regarding data privacy. In contrast, TinyML-based solutions focus on enabling low-latency, always on voice recognition on ultra-low-power micro-controllers by deploying quantized neural network models locally on edge devices.

Several TinyML applications have successfully deployed lightweight models for keyword spotting on constrained hardware. For instance, Guaman et al. [5] implemented an Alexa-based home automation prototype but faced challenges with latency and privacy due to its cloud dependency. Choi et al. [14] achieved a significant 385× speedup using a ResNet-based model optimized for mobile deployment. Similarly, Mittermaier et al. [18] designed an energy-efficient CNN using SincConvs and achieved 96.4% accuracy with 62K parameters. He et al. [26] proposed TENet on an FPGA with 95.36% accuracy. Ng et al. [27] reached a 98.2% accuracy with a ResNet-15 model targeting noisy environments but with higher memory consumption (~100K parameters).

The proposed system in this study distinguishes itself by combining DCNN with MFCC feature extraction and SpecAugment based augmentation as well as applying post-training quantization for edge deployment. It achieves 96.67% accuracy with only 19.8 KB RAM and 11 ms latency which significantly outperform many of the referenced methods in terms of resource efficiency. This compact and high-performing implementation on an Arduino Nano BLE Sense demonstrates the practical viability of TinyML for real-time, private, and energy-efficient voice-controlled home automation in constrained environments.

3 Methodology

The development of the compact keyword spotting model involves the steps described in Fig. 2. The features of the voice data are extracted using MFCC. The extracted features are then used as input to a DCNN designed to learn and identify patterns in the data. The model is trained on the voice data and corresponding labels (e.g., spoken words) to create a model that can recognize voice commands. Once the model is trained, it is converted into a smaller, efficient format using quantization, reducing its size and making it suitable for deployment on resource-constrained devices like micro-controllers. The trained and quantized model is then converted into a C/C++ format so that it can be loaded and executed on small, embedded devices like microcontrollers. The TinyML interpreter is responsible for implementing the model on the device. It uses the device's limited memory to process input data and run the inference.

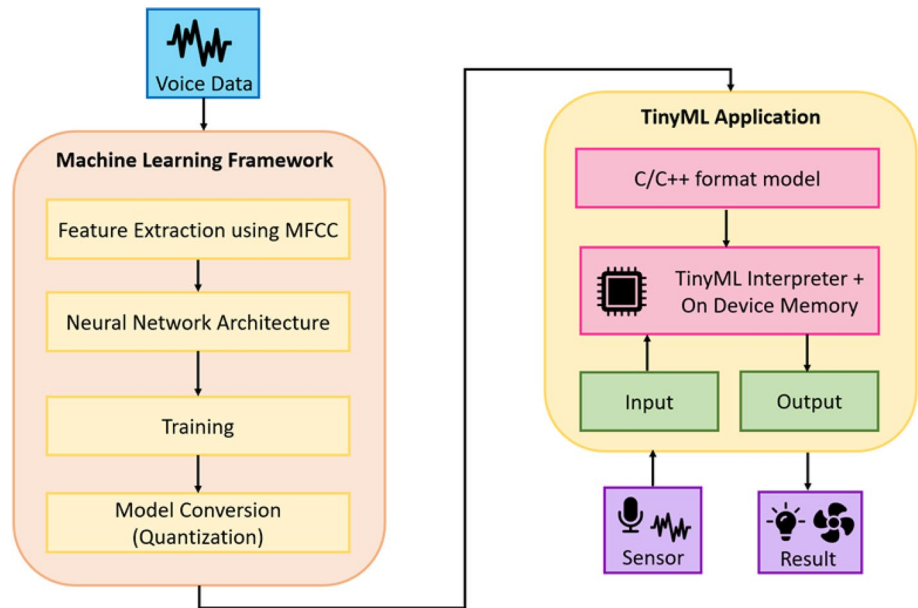
The input to the system comes from a microphone that captures voice data, which is fed into the TinyML model for inference. After processing the input, the model produces an output, such as recognizing a specific word or command. This output can trigger an action, like turning on a light or controlling a device. The whole pipeline represents a TinyML workflow for voice recognition, starting from raw data collection through feature extraction, training, and deployment onto a TinyML-supported device.

3.1 Data collection

The real-world audio datasets comprising voice instructions important to home automation operations is collected. To ensure the model's adaptability, the dataset comprises a variety of speakers, settings, and noise circumstances. Obtaining labelled data for machine learning is a critical step in developing accurate models. To classify the data in this case, three separate classes “light_on,” “light_off,” and “nocmd” have been used. These labels represent operations such as turning on and off the lights, as well as times when no specific instruction is intended. However, extra preprocessing is required for data to be genuinely viable for machine learning.

Table 1 Comparison of keyword spotting methods

Author	Feature used	Method	Contribution	Platform	Dataset
Guaman et al. [5]	Multiple keywords	Alexa	Improved Latency	Cloud Based	Pretrained Model
Chen et al. [11]	Multiple keywords	DNN, HMM	Accuracy improvement	Edge AI Based	Speech Corpus
Seungwoo Choi et al. [14]	Multiple keywords	ResNet, CNN	Faster inference	Edge AI Based	Google Speech Commands dataset
Leroy et al. [15]	Hey Snips	MFCC, CNN	Accuracy improved	Edge AI Based	Custom Dataset
Theodore Bluche et al. [16]	Multiple keywords	TDNN-LSTM, Greedy Approach	Not mentioned	Edge AI Based	Librispeech dataset
Hard et al. [17]	Multiple keywords	Encoder-decoder, SVDF layers	Reduces false rate	Edge AI Based	Custom dataset
Mittermaier et al. [18]	Multiple keywords	ResNet, CNN	Improved accuracy	Edge AI Based	Google Speech Commands dataset
Rybakov et al. [19]	Multiple keywords	DNN, CNN, RNN	Improved classification	Edge AI Based	Google Speech Commands dataset
Rafael Stahl et al. [20]	Not available	CNN, VGG-16, GoogLeNet	Improved inference	Edge AI Based	Custom Dataset
Iván López-Espejo et al. [21]	Multiple keywords	DKWS, MFCC	Improved accuracy	Edge AI Based	Google Speech Commands dataset
Zuozhen Liu et al. [22]	Open vocabulary	RNN-Transducer	Improved latency	Edge based	Custom Dataset
Shrivastava et al. [23]	Multiple keywords	MFCC, DNN-HMM, CNN	Improved classification accuracy	Edge AI Based	Custom Dataset
Norah N. Alajlan et al. [25]	Multiple keywords	MobileNet-V1	Improved accuracy	Edge AI Based	Google Speech Commands dataset
Keyan He et al. [26]	Multiple keywords	TENet, MFCC, CNN, TCN	Improved accuracy	Edge AI Based	Google Speech Commands dataset
Dianwen Ng et al. [27]	Multiple keywords	ResNet-15, RNN	Improved accuracy	Edge based	Google Speech Commands V2
Ahmed et al. [28]	Multiple keywords	MFCC, DNN	Improved accuracy	Edge AI Based	Custom Dataset
Proposed System	Light On, Light Off	MFCC, DNN	Improved accuracy, latency	Edge AI Based	Custom Dataset

Fig. 2 Workflow of KWS

To maintain accuracy and avoid misclassification, the audio data segments containing the real keywords must be properly extracted. This extraction entails separating the sections where the term is pronounced while removing any background noise or irrelevant speech. By completing this critical step, the labelled data becomes more focused and targeted, displaying only the instances when the desired term is utilised.

The following equation expresses data segmentation using a moving window technique in which a specified stride is used to move the window across the data (as shown in Fig. 3).

- The dataset is represented by D
- A segment of data is represented by S_i , where i represents the i -th segment
- The number of data points the window moves is represented by s

Therefore, the equation for data segmentation using a moving window will be:

$$S_i = D[is : is + w] \quad (2)$$

Where:

- $D[is : is + w]$ is a portion of the dataset D , with starting index is till end index $is + w$.
- w represents the count of data points in the duration of the moving window.

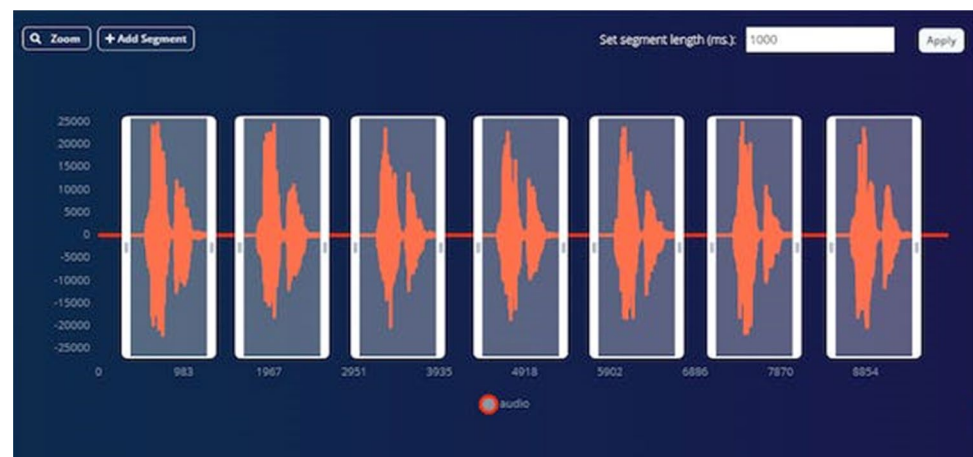
Fig. 3 Audio data segmentation

Fig. 4 Audio data segmentation**Table 2** Overview of the collected dataset

Description	Value
Number of participants	15
Participant demographics	Mixed age and gender
Language	English
Commands recorded	"light on", "light off", "nocmd"
Total audio samples	12,300
Sampling rate	16 kHz
Clip duration	1 s
Environment	Real-world (home settings)

After splitting the dataset, the dataset for all three labels is shown in Fig. 4.

Despite the fact that the data was initially labelled with appropriate classes, the efficacy of machine learning models is dependent on effectively separating keyword occurrences. The data is refined by methodically isolating these segments, laying the groundwork for the construction of high-performance voice recognition models. This method guarantees that the models are carefully trained to recognise the keyword, reducing the possibility of misclassification and improving the system's overall accuracy and dependability. The dataset used in this study was collected from 15 voluntary participants belonging to different age groups and both genders within Indian households. Each participant recorded voice commands in English, including the target phrases "light on", "light off", and instances with no specific command ("nocmd"), under real-world ambient conditions such as background chatter, fan noise, and varied room acoustics. The raw recordings were captured at a 16 kHz sampling rate and subsequently segmented into fixed-duration audio clips of 1 s each using a sliding window approach with a stride of 0.5 s. The final curated dataset consists of a total of 12,300 labeled audio samples. To ensure high-quality and balanced data, each class was equally represented, and irrelevant segments or clips with excessive background noise were filtered out.

During preprocessing, MFCC [30] were extracted from each clip using a frame length of 0.02 s, frame stride of 0.02 s, and 13 coefficients per frame. Additional preprocessing steps included the application of SpecAugment [31] to improve model robustness across varied speaking styles and noise conditions. The resulting dataset preserves key acoustic features necessary for reliable real-time voice command recognition in smart home environments. The summary of the collected dataset is represented in Table 2, and the class-wise distribution of the audio samples is shown in Table 3.

Table 3 Class-wise distribution of audio samples

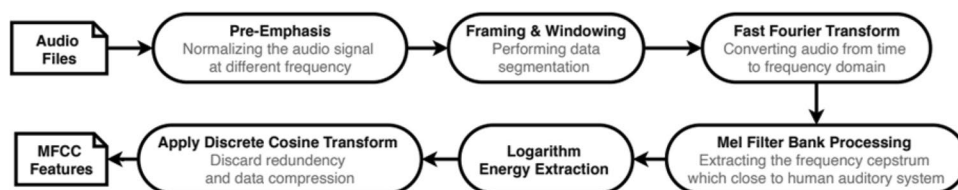
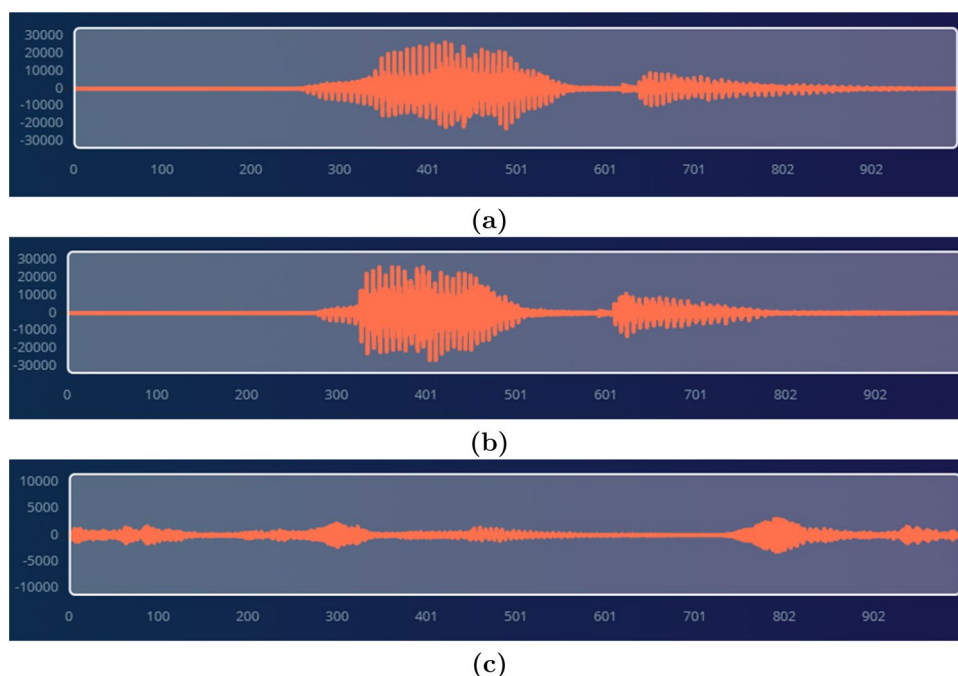
Command class	Number of samples
"light on"	4100
"light off"	4100
"nocmd"	4100
Total	12,300

3.2 Preprocessing

In this study, MFCC is used to extract relevant characteristics from audio. MFCC, a prevalent audio file pre-processing technique for speech-related classification applications [30, 32–36], involves different blocks within its pipeline. These blocks are used to extract speech data, considering the nuances of the human voice. Fig. 5 illustrates the MFCC pipeline [37].

The basic steps used in MFCC-based preprocessing are as follows:

- **Preprocessing and Framing:** The audio signal is first pre-emphasized to boost high frequencies, then segmented into overlapping frames to capture short-time spectral properties.
- **Windowing and Fourier Transform:** Each frame is windowed (usually with a Hamming window) to reduce spectral leakage. A Fourier transform is then applied to convert the time-domain signal into the frequency domain.
- **Mel Filter Bank Processing:** The power spectrum of each frame is passed through a series of filters spaced according to the Mel scale, which approximates human auditory perception. This step emphasizes frequencies that are more important to how we hear.

Fig. 5 MFCC pipeline**Fig. 6** Raw data. **a** Lights-On Class, **b** Lights-Off Class, **c** Nocmd Class

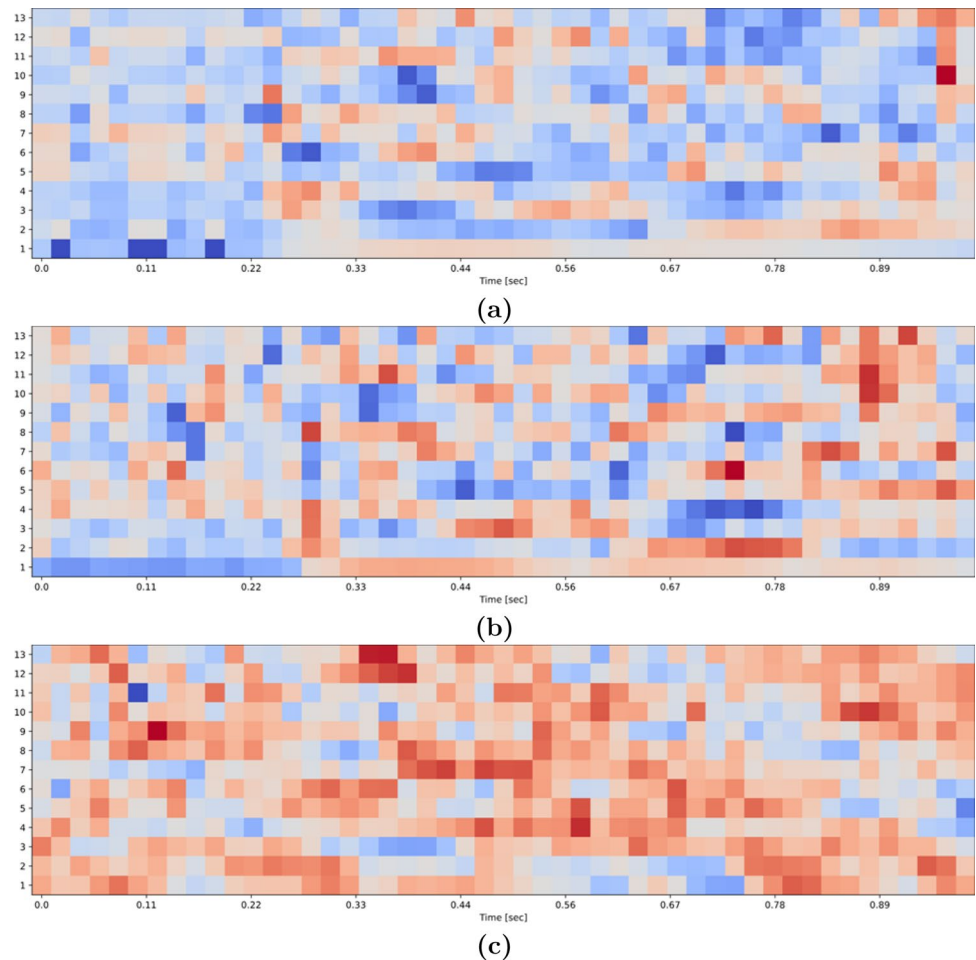
- **Logarithmic Compression and DCT:** The logarithm of the filter bank energies is taken to simulate the logarithmic response of the human ear to signal amplitude. Finally, a Discrete Cosine Transform (DCT) is applied to decorrelate the features, resulting in the MFCCs that represent the spectral properties of the audio in a compact form.

This approach improves the recognition of voice commands, as it is particularly good at capturing human voice characteristics. The dataset, shown in Fig. 6, gathered for this purpose, acts as the basis, with MFCC working as a tool for extracting important information from speech signals, which is required in subsequent recognition tasks.

Table 4 Mel frequency cepstral coefficients

Parameters	Value
Number of coefficients	13
Frame length	0.02 s
Frame stride	0.02 s
Filter number	32
FFT length	256
Normalization window size	101
Low frequency	300 Hz
High frequency	0 Hz
Coefficient	0.98
Shift	1

Fig. 7 Cepstral coefficients. **a** Lights-On Class, **b** Lights-Off Class, **c** Nocmd Class



The parameters used to extract features using MFCC from audio data are presented in Table 4, and the output of the MFCC pipeline, representing the spectral characteristics as Cepstral Coefficients of the audio data is visualized in Fig. 7. This procedure culminates in the development of the “processed features” array. The processed features are the final, numeric representation of audio data passed input to the machine-learning model. These features are created when the raw waveform is passed through the MFCC pipeline. Each audio frame ends up with a set of cepstral coefficients, and these values are flattened into a single vector, the processed features. This numeric vector captures the essential characteristics of the sound in a compact form that the model can learn from Audio MFCC [38].

Following is the Mel Frequency Cepstral Coefficients where

- Number of coefficients: The number of cepstral coefficients to retain Cosine Discrete Transform.
- Frame length (in seconds): The length of each frame.
- Frame stride: The time it takes between frames.
- Filter Number: The number of triangle filters used to filter the spectrogram.
- FFT time: The FFT dimension
- Low frequency: Mel-scale filterbanks’ lowest band edge
- High frequency: Mel-scale filterbanks’ highest band edge
- Window size: The size of the sliding window for normalising the local cepstral mean. The size of the windows must be unusual.
- Coefficient: The pre-emphasizing coefficient that will be applied to the input signal (0 means no filtering).

The Fig. 8 show the generated features.

3.3 Model architecture

A lightweight deep learning architecture for keyword detection is created. While retaining appropriate accuracy levels, the model architecture prioritises efficiency and low computing complexity. The architecture also consists of data augmentation for spectrograms known as SpecAugment. SpecAugment is a simple data augmentation technique used in the tasks like voice recognition. SpecAugment is directly applied to the input features of a neural network. The augmentation involves warping the features, masking blocks of frequency channels, and masking blocks of time steps.

The SpecAugment [31] is immediately applied to the log mel spectrogram to improve the network’s learning of valuable features. The goal is to guarantee that these traits resist time direction deformations, partial loss of frequency information and tiny speech segments. The following Figs. 9 and 10 show extracted Mel Spectrogram Features from the audio file.

Applying SpecAugment for time wrapping.

Fig. 8 Feature explorer representing the three classes

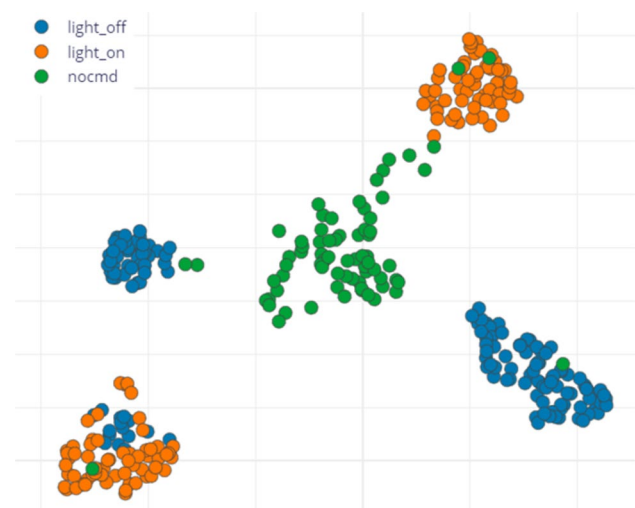
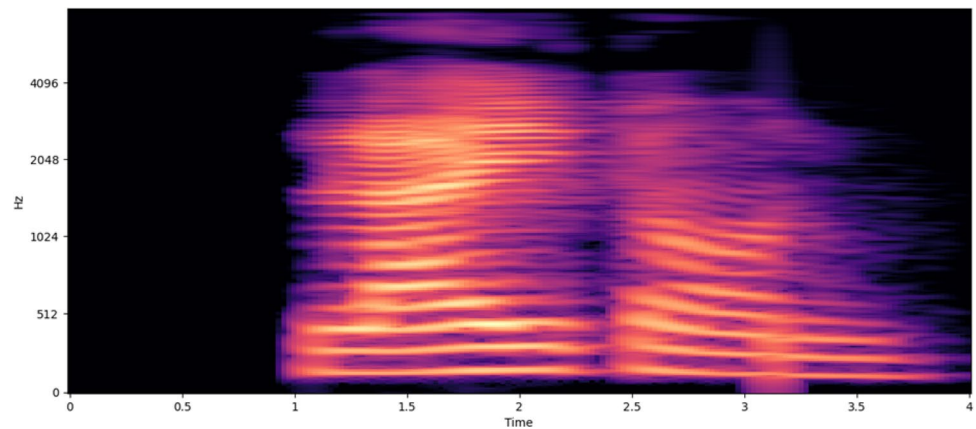
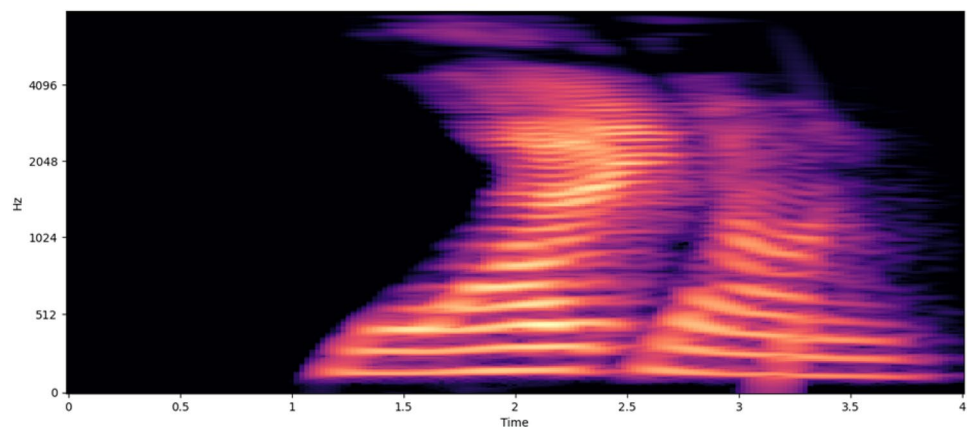


Fig. 9 Mel spectrogram features**Fig. 10** Time wrapping

Applying SpecAugment for time wrapping



The DCNN design used in this study is made up of multiple separate layers, each of which contributes to the thorough processing of incoming data. The neural network architecture is illustrated in Fig. 11. The input receives data with dimensions $49 \times 13 \times 1$. The first layer is a 2D convolutional layer with 8 filters, each of size 3×3 , applied to the input. This layer outputs 8 feature maps of size 49×13 , followed by a ReLU activation function to introduce non-linearity.

Next, a MaxPooling layer with a pooling size of 2×2 reduces the spatial dimensions to 25×7 , while maintaining the depth of 8 channels. The second convolutional layer follows, with 16 filters of size 3×3 , expanding the depth to 16 feature maps, which are again reduced in spatial dimensions to 13×4 after another MaxPooling layer.

The feature maps are then reshaped from a 4D tensor of size $1 \times 13 \times 4 \times 16$ to a 2D tensor of size 1×832 , flattening the spatial dimensions and concatenating all channels. This flattened vector is passed to a fully connected (dense) layer with 3 output neurons, corresponding to 3 classes. The ReLU activation function is applied again in this dense layer, followed by a Softmax layer, which produces a probability distribution across the 3 output classes. The final output is a 1×3 vector representing the predicted class probabilities.

The architecture uses Adam Optimizer, a very efficient approach for altering model weights during training. The categorical cross-entropy is used as the loss function, which is appropriate for multi-class classification applications. This combination of architecture, optimizer, and loss function offers a methodical and effective approach to harnessing complex patterns in the input data, resulting in reliable classification outputs.

DCNNs efficiently process input speech spectrograms through convolutional layers, which can run in parallel across multiple filters and significantly reduce computational overhead. The proposed DCNN is highly optimized for low-power and memory-constrained devices. DCNN-based KWS models consume significantly lower power than RNNs or Transformers based systems due to fewer parameters and efficient operations. Many recent studies and real-world TinyML applications favour DCNNs for KWS due to their balance of accuracy, efficiency, and deployability on resource-constrained devices like mobile phones and microcontrollers, optimizing for speed, size, and power efficiency [39, 40].

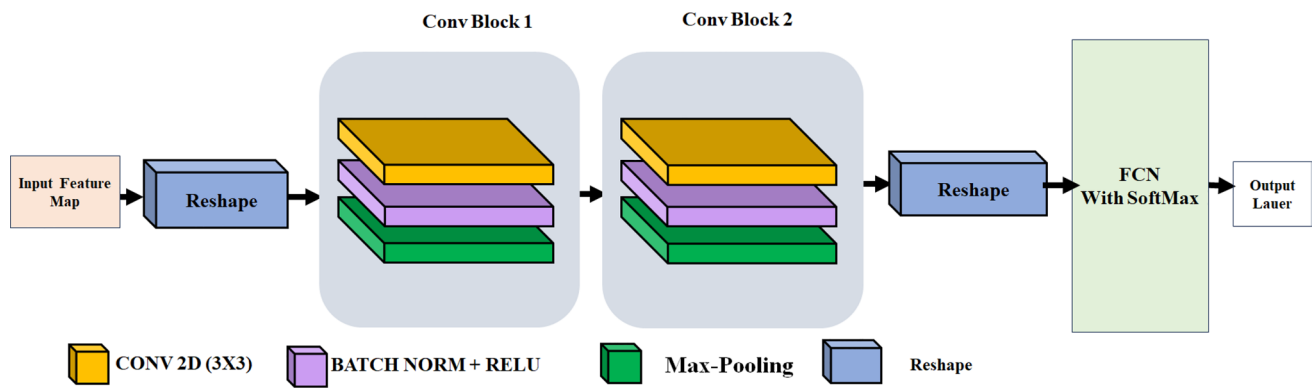


Fig. 11 proposed DCNN architecture

Fig. 12 Data labels (ground truth)

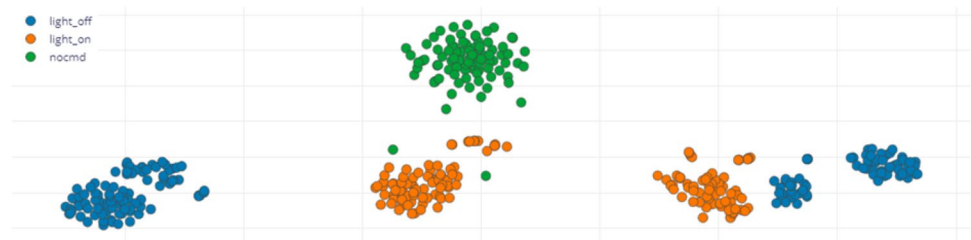
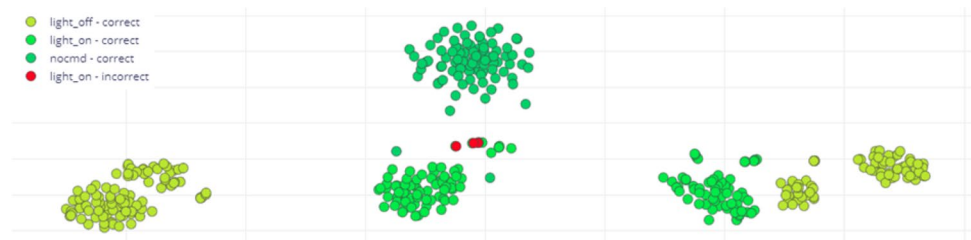


Fig. 13 Predictions



The implementation of SpecAugment [31, 41] improves the model's ability to generalize across different voice commands and noisy environments by introducing controlled distortions in the spectrograms during training. By applying

Fig. 14 Confusion matrix (validation dataset)

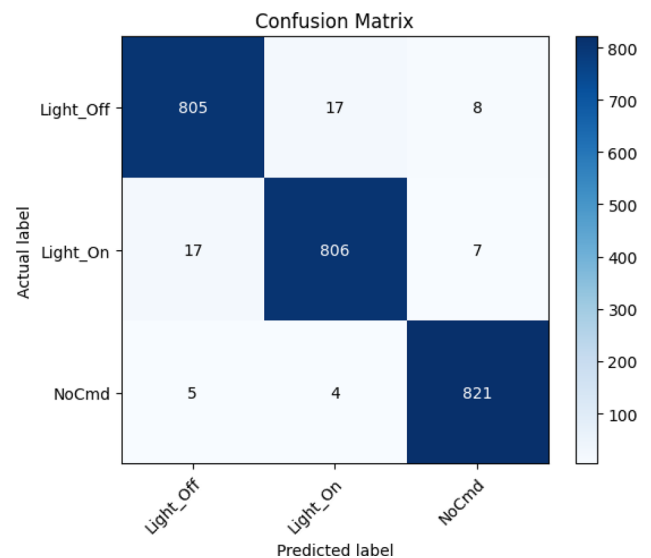
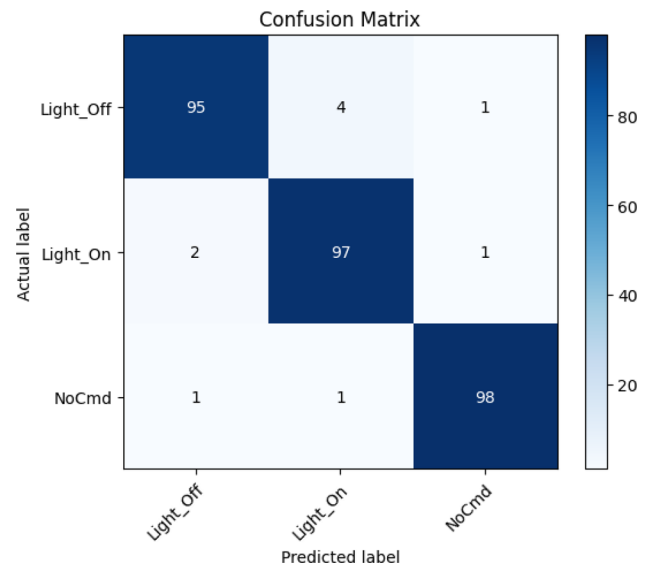


Fig. 15 Confusion matrix (test dataset)



techniques like time warping, frequency masking, and time masking, SpecAugment forces the model to learn more robust and invariant features rather than memorizing specific patterns. This helps the model handle variations in speaker accents, background noise, and recording conditions. As a result, the trained model becomes more adaptable to real-world scenarios, improving its accuracy and robustness to recognize voice commands even in challenging environments.

4 Training and evaluation of DCNN

A structured training methodology is used in this study, encompassing data preparation, feature extraction, model design, and performance evaluation to develop and deploy the proposed DCNN model on low-resource IoT devices.

4.1 Performance evaluation metric and training configuration

The performance of the proposed model is evaluated on following metrics:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

$$Precision = \frac{TP}{TP + FP} \quad (4)$$

Fig. 16 Data explorer for training results

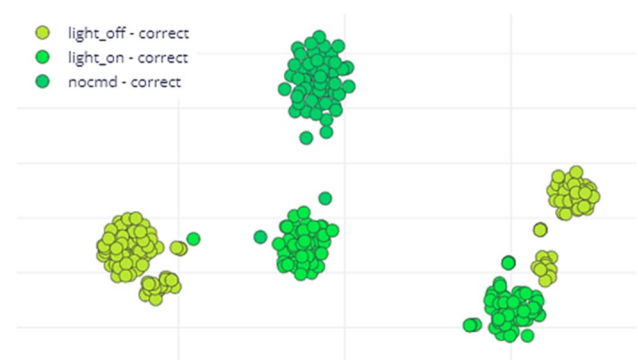
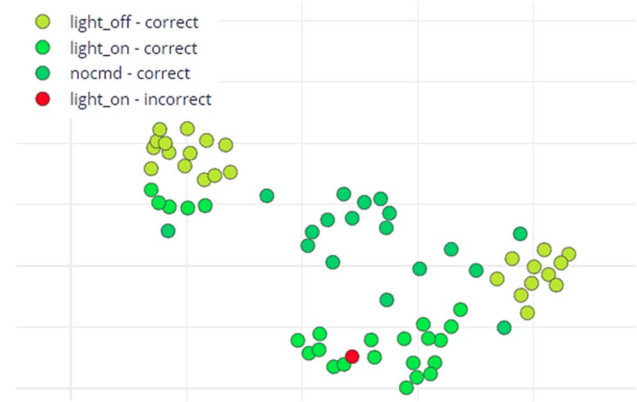


Fig. 17 Data explorer for validation results**Table 5** Evaluation metrics on validation data

	LIGHT_OFF	LIGHT_ON	NOCMD
LIGHT_OFF	96.99%	2.05%	0.96%
LIGHT_ON	2.05%	97.11%	0.84%
NOCMD	0.60%	0.48%	98.92%
Precision	0.97	0.97	0.98
Recall	0.97	0.97	0.99
F1 SCORE	0.97	0.97	0.99
Accuracy	97.67%		

Table 6 Evaluation metrics on testdata

	LIGHT_OFF	LIGHT_ON	NOCMD
LIGHT_OFF	95.00%	4.0%	1.0%
LIGHT_ON	2.0%	97.0%	1.0%
NOCMD	1.0%	1.0%	98.0%
Precision	0.97	0.95	0.98
Recall	0.95	0.97	0.98
F1 SCORE	0.96	0.96	0.98
Accuracy	96.67%		

$$Recall = \frac{TP}{TP + FN} \quad (5)$$

$$F1Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (6)$$

To train the model, audio data set of users voice are collected. Quantization and pruning techniques are applied to optimize the model for resource-constrained devices. During the training phase, particular settings were used to fine-tune the neural network model. The procedure consisted of 100 epochs, each representing a complete iteration of the training dataset. This allowed the model to gradually tune its parameters and comprehend the underlying patterns in the data. In this study, the learning rate of 0.005 is used. This parameter affects the magnitude of weight modifications made during training, allowing the model to converge towards ideal accuracy without overshooting. The selected learning rate achieves a compromise between quick convergence and stability, contributing to the robustness of the training process. The training dataset was divided into batches for each epoch, each with 32 data samples. The batch size optimizes memory utilization and computational performance during training. It enables the model to repeatedly

Table 7 Arduino nano 33 BLE sense specifications

Specification	Details
Processor	nRF52840
Flash Memory	1 MB
SRAM	256 KB
Operating Voltage	3.3 V
Input Voltage (limit)	21 V
Digital I/O Pins	14
PWM Pins	All digital pins
Analog Input Pins	8 (12-bit ADC)
Communication Interfaces	UART, SPI, I ² C
IMU	LSM9DS1
Microphone	MP34DT05
Gesture, light, proximity sensors	APDS9960
Barometric pressure sensor	LPS22HB
Temperature, humidity sensor	HTS221

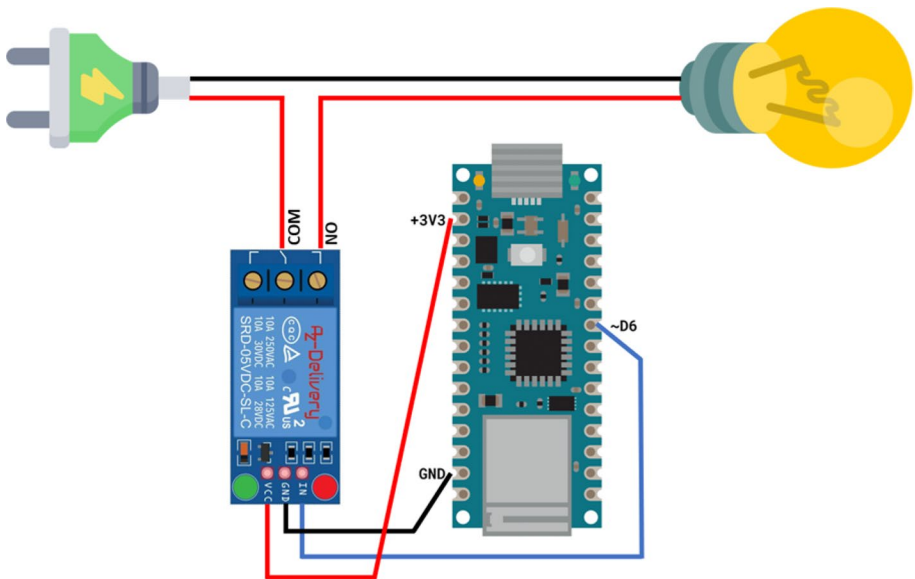
update its weights and biases depending on a subset of the data, increasing total training speed. The following metrics were used to evaluate the performance of the proposed model.

The proposed model achieved a test accuracy of 96.67%, with precision, recall, and F1-scores above 95% for all classes. Confusion matrices for both validation and test sets are shown in Figs. 14 and 15 confirming strong generalization across commands.

The trained model is evaluated using a separate test dataset, and the ground truth of the test data is shown in Fig. 12. The predicted results are shown in Fig. 13, which demonstrates that there are few samples that are misclassified. Figs. 14 and 15 illustrate the confusion matrix for validation and test sets. The model's accuracy, memory footprint, and processing speed are measured on resource-constrained IoT devices, and the experimental results are visualized in Figs. 16 and 17, showing the ground truth and the prediction by the model respectively.

The performance of the designed DCNN model is given in Tables 5 and 6. The model achieved an overall training accuracy of 97.67% and test accuracy of 96.67%, and F1 scores for each class, i.e. Lights-On, Lights-Off, and Nocmd indicate remarkable performance. The following Fig. 15 depicts all data in the training set classified by the DNN. The green dots specify all correct classification while red specify incorrect classification. There is no misclassification in the training set by the model hence no red dots are present indicating how well the model has performed during training.

Fig. 18 Circuit diagram



5 Experimental setup and result discussion

In this section, the deployment of the trained DCNN model using NN Quantization approach is discussed along with experimental results.

5.1 NN quantization

Traditionally, neural network training utilizes floating-point arithmetic for weights and activations. However, recent research [4, 42–44] has demonstrated that fixed-point representations can be used with minimal impact on accuracy. This finding is particularly relevant for microcontroller-based systems facing significant memory limitations. To deploy the DCNN models on such hardware, it is often necessary to convert weights from 32-bit floating point to 8-bit fixed point format. This quantization process substantially reduces the model's memory footprint, making it suitable for resource-constrained microcontrollers. This method is faster because microcontrollers can perform fixed-point calculations more quickly than floating-point calculations.

The quantization methods described in [43] is used in this research, with 8 bits representing both weights and activations. The value v of a 2's complement 8-bit fixed point number may be expressed as

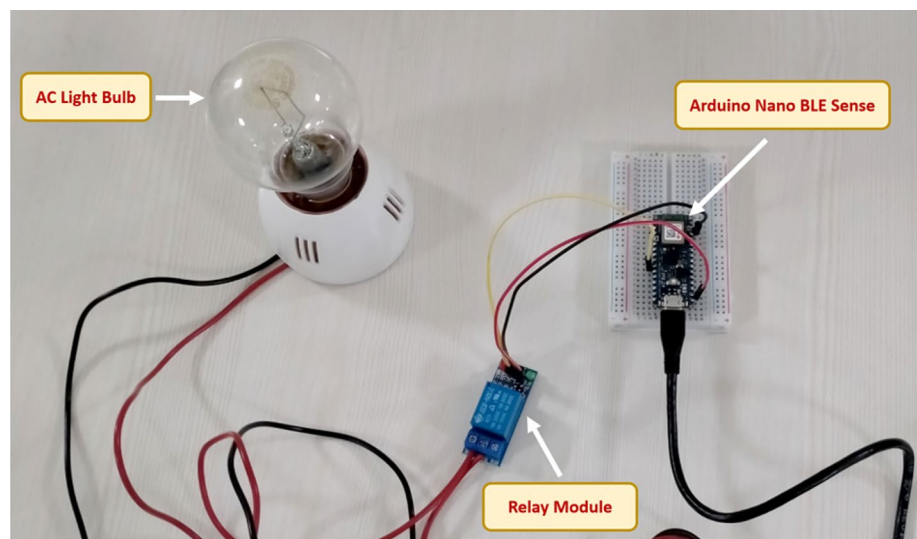
$$v = -B_7 \cdot 2^{7N} + \sum_{i=0}^6 B_i \cdot 2^{i-N} \quad (7)$$

where N indicates the negative fractional length. N remains constant inside a given layer but varies between layers. When $N = 0$, it represents the range from -128 to 127 with step 1. Similarly, $N = 7$ represents range from 1 to 1 ($1/27$) with a step of $1/27$. However, $N = 2$ represents range from -512 to 508 with a step of 22. The quantization process involves a layer-by-layer reduction of weights to 8-bit precision. This is achieved by determining the optimal fractional length N for each layer, aiming to minimize accuracy loss from quantization. After quantizing all weights, the method shifts to activations. A parallel approach is applied to quantize activations, identifying the applicable fractional length N for each layer simultaneously. This two-phase strategy allows precise control over the quantization process, balancing preservation accuracy with reduced computational demands.

5.2 Deployment and result discussion

The proposed voice-controlled home automation system is deployed on the Arduino Nano 33 BLE Sense board, which is engineered for low-power, edge AI applications. The board is powered by the Nordic nRF52840 System-on-Chip (SoC). A

Fig. 19 Experimental setup for testing of KWS TinyML model



detailed specification is given in Table 7. These specifications necessitate the use of optimized, quantized neural network architectures to ensure real-time inference within the limited memory and processing capabilities.

In terms of power efficiency, the board supports various low-power modes. By disabling non-essential peripherals and utilizing sleep modes, the current consumption can be significantly reduced. For example, implementing deep sleep states can lower the current draw to approximately 1.0 μA [45]. During active inference, the current consumption is around 4 mA, depending on the specific workload and peripheral usage. These power characteristics make the Arduino Nano 33 BLE Sense well-suited for battery-operated, always-on voice recognition systems in smart home environments. To further enhance energy efficiency, board's built-in sensors may be used only when required by writing code efficiently which can help IoT devices last longer on a single battery charge.

The circuit diagram of the system deployment is shown in Fig. 18 and experimental setup in Fig. 19. The experiments are conducted with different audio input scenarios, including various voice commands, background noises, and room acoustics.

The image depicts the experimental setup for testing the KWS model on target device. A microcontroller board is connected to a relay module, which in turn controls the power supply to a light bulb. The microcontroller board runs the TinyML KWS model. A built-in microphone on the microcontroller board allows to capture audio input for keyword spotting. This setup allows for real-time voice control of the light bulb, where specific keywords can trigger the device to turn the light on or off.

The results demonstrate that the proposed model achieves high accuracy in recognizing voice commands associated with home automation tasks. Furthermore, the model's memory footprint and computational requirements are significantly reduced, allowing it to run seamlessly on resource-constrained IoT devices.

A thorough analysis on input data, alternative signal processing blocks, and neural network structures is conducted in this research. It provides a complete overview of feasible model architectures which are best suited for the target device. Following parameters are used to identify which model architecture is most suitable for the target device:

- **Input:** Several experiments with various sample window sizes and step sizes are conducted. A window size of 1000 ms with 500 ms increment at 16,000 Hz frequency was taken.
- **Processing Blocks:** For signal processing, DSP type as MFCC with different frame length, frame stride and number of filters in filterbank is used.
- **Learning Blocks:** The machine learning phase involved testing different neural network architectures and hyperparameters. Various data augmentation techniques to improve model robustness are tried. The tests covered a wide range of combinations, adjusting both the preprocessing steps and network structures. Finally, the overall performance comparison was done to find the best configuration.

The objective of this research is to find the best model architecture that fits the requirements of the target device. This entails a thorough examination of architectural and configuration options to ensure maximum performance and effectiveness while installing the TinyML model. The following table illustrates the different combinations tested to determine the most suitable model for the intended device. Extensive testing and evaluation were carried out, with numerous model designs, setups, and parameters being investigated.

Finally, the best model for device deployment was chosen without sacrificing accuracy. Tables 8 and 9 compare the proposed solution with the existing solutions and Table 10 compares the converted TensorFlow Lite model to the quantized model using post-training quantization technique.

The performance of the proposed method is compared with existing approaches (e.g., DNN, CNN, ResNet, etc.) [5, 11, 14–20, 22, 23, 25–28] deployed on either edge or cloud platforms as presented in Tables 8 and 9. While the existing solutions show decent accuracy and latency improvements, they often face limitations such as high computation and memory usage, network dependency, or limited scalability. In contrast, the proposed model achieves a 96.67% improvement in accuracy and significantly enhances computation and memory management. This not only reduces the overall resource footprint but also delivers better real-time performance compared to prior methods.

In the Table 10, it is seen that the model is quantized to fit the resource constraint device without losing accuracy. The outcomes of the study verify the efficacy of the suggested keyword spotting model specifically designed for edge devices in the IoT infrastructure. The model overcomes the limitations of memory usage, processing capacity, and energy consumption by employing TinyML approaches, which make it a viable option for voice-controlled home automation in such contexts. Implementing the keyword spotting model directly on IoT devices reduces the requirement for constant internet access while also addressing privacy concerns associated with data transfer to other servers. This localization

Table 8 Benchmark comparison of proposed system with existing voice-activated automation methods

System	Architecture	Deployment	Accuracy	Latency	Memory usage	Internet required	Privacy
Guaman et al. [5]	Alexa Cloud System	Cloud	~97%	High	High	Yes	Low (cloud data)
Chen et al. [11]	DNN + HMM	Edge	~95%	Moderate	Moderate	No	High
Choi et al. [14]	ResNet + Temporal CNN	Edge	~95.3%	~2–3 ms	Moderate	No	High
Leroy et al. [15]	MFCC + CNN (Federated)	Edge	95%	Moderate	Low	No	High
Mittermaier et al. [18]	SincConv + GDSConv	Edge	96.4%	~12–14 ms	62K params	No	High
He et al. [26]	TENet + Simplified MFCC	FPGA/Edge	95.36%	Low	FPGA optimized	No	High
Ng et al. [27]	ResNet-15 + ConvMixer	Edge (Noisy Env)	98.2%	Moderate	~100K params	No	High
Ahmed et al. [28]	MFCC + DNN (EKOS)	Edge	83%	Moderate	Not specified	No	High
Proposed system	DCNN + MFCC + SpecAugment	Edge (TinyML)	96.67%	11 ms	Low (20KB RAM)	No	High (on-device)

Table 9 Comparison of existing methods with proposed method

Author	Platform	Techniques	Observation
Guaman et al. [5]	Cloud	Alexa	Improved Latency
Chen et al. [11]	Edge	DNN, HMM	45% relative improvement
Choi et al. [14]	Edge	ResNet, CNN	385x speedup, 0.3%p increase
Leroy et al. [15]	Edge	MFCC, CNN	Accuracy 95%
Bluche et al. [16]	Edge	TDNN-LSTM, Greedy Approach	Improved accuracy
Hard et al. [17]	Edge	Encoder-decoder, SVDF layers	Reduces false rate by 56%
Mittermaier et al. [18]	Edge	ResNet, CNN	Accuracy 96.4%
Rybakov et al. [19]	Edge	DNN, CNN, RNN	Improved classification error by 10%
Stahl et al. [20]	Edge	CNN, VGG-16, GoogLeNet	Improved inference, High computation requirement
López-Espejo et al. [21]	Edge	DKWS, MFCC	Improved accuracy
Liu et al. [22]	Edge	RNN-Transducer	Improved FPR 3.70%
Shrivastava et al. [23]	Edge	MFCC, DNN-HMM, CNN	Avg. classification accuracy 68.1%
Alajlan et al. [25]	Edge	MobileNet-V1	Highest Accuracy 96.4%
He et al. [26]	Edge	TENet, MFCC, CNN, TCN	Accuracy 95.36%
Ng et al. [27]	Edge	ResNet-15, RNN	Accuracy 98.2%
Ahmed et al. [28]	Edge	MFCC, DNN	Accuracy 83%
Proposed system	Edge	MFCC, DNN	Accuracy 96.67%, improved latency, improved memory management

Table 10 Comparison of unoptimized and quantized models (on test dataset)

	Unoptimized model		Quantized model	
	Classifier	Total	Classifier	Total
Latency	18 ms	18 ms	11 ms	11 ms
RAM	66.0 K	66.0 K	19.8 K	19.8 K
Flash	119.1 K	–	55.0 K	–
Accuracy	96.67%		96.67%	

of speech recognition improves the overall user experience and guarantees that the system operates even without internet connection.

5.3 User testing and feedback

To evaluate the practical usability and effectiveness of the proposed voice-controlled home automation system, a user study was conducted involving 15 participants (who were involved in data collection) from diverse age groups and backgrounds. Each participant interacted with the system in a controlled home environment, performing tasks such as turning lights on and off using voice commands. The feedback summary is given in Tables 11 and 12.

Participants reported satisfaction with the system's responsiveness and accuracy. The average command recognition accuracy observed during the user testing was 93.5%, closely aligning with the system's benchmark performance. Users appreciated the on-device processing, noting the quick response times and the absence of reliance on internet connectivity, which also alleviated privacy concerns.

Some participants suggested enhancements such as the inclusion of additional voice commands and support for regional languages to cater to a broader user base. These insights are valuable for guiding future iterations of the system.

The positive feedback and high usability ratings indicate that the system is well-received by users and holds promise for real-world deployment in smart home environments.

Table 11 User testing feedback summary

Feedback aspect	Average rating (1–5)
Ease of use	4.7
Responsiveness	4.1
Accuracy of voice Recognition	4.6
Satisfaction with system performance	4.0
Likelihood to use in daily life	4.5

Table 12 User feedback dataset (ratings: 1 = Poor, 5 = Excellent)

Participant ID	Ease of use	Responsive-ness	Accuracy	Satisfaction	Daily use intent
P1	5	4	5	4	5
P2	4	3	5	4	5
P3	5	4	4	3	4
P4	5	5	5	4	5
P5	4	4	4	4	4
P6	4	3	5	3	4
P7	5	5	5	5	5
P8	5	3	4	3	5
P9	4	4	4	4	4
P10	5	4	5	4	5
P11	4	4	4	4	4
P12	5	5	5	5	5
P13	4	3	4	3	4
P14	5	4	5	4	5
P15	5	4	5	4	5
Average	4.7	4.1	4.6	4.0	4.5

5.4 Security considerations in voice-activated IoT systems

Voice-activated systems, such as home automation systems discussed in this study, interact with critical devices such as lights, door locks, and appliances, making them vulnerable to several security threats. These include spoofing attacks (e.g., playing pre-recorded or synthetic voice commands) [46], adversarial perturbations that fool the model [47], and privacy concerns related to audio data capture. To overcome such risks, the proposed system addresses key vulnerabilities in the following ways:

- **On-device inference:** All voice processing is performed locally on the microcontroller, ensuring that no voice data is transmitted to external servers therefore the system protects user privacy and reduce exposure to network based attacks. The system only sends status information (i.e whether the device is on or off and usage details) to cloud servers for generating reports and other statistics.
- **Always-on security filtering:** Using lightweight models with limited command vocabulary, the system minimizes the risk of unintended activation and reduces the attack surface.
- **Model quantization:** Besides reducing memory and computation requirements, quantized models are less susceptible to certain adversarial manipulations due to reduced precision and numerical sensitivity.
- **Future enhancements:** Additional security layers such as voice authentication, replay attack detection, and encrypted firmware can be integrated to protect the system against evolving threats.

By localizing computation and avoiding cloud dependency, the proposed solution inherently improves data confidentiality, reduces latency, and enhances user trust, making it suitable for privacy-sensitive environments like smart homes and healthcare settings.

6 Conclusion

This study demonstrates the feasibility and effectiveness of deploying a lightweight, real-time voice-controlled home automation system using TinyML. The study uses MFCC for feature extraction, SpecAugment for data augmentation, and a quantized DCNN. The proposed solution achieves an accuracy of 96.67% while maintaining the low memory usage of 19.8KB RAM and fast inference latency of 11ms. The system operates entirely on-device, which makes it suitable for privacy-preserving, low-power IoT applications. Key contributions of this research include:

- Development of a compact and quantized DCNN model optimized for microcontroller deployment.
- Demonstration of robust keyword spotting across diverse acoustic conditions using SpecAugment.
- Successful deployment on an Arduino Nano BLE Sense board with low computational and memory footprint.

These results establish a practical foundation for integrating voice-based control into resource-constrained smart environments, such as energy-efficient homes, industrial monitoring systems, and ambient health interfaces. While the system performs effectively for a limited set of English commands, its robustness can be further improved by:

- Expanding the dataset to include multilingual and multi-accent voice commands.
- Incorporating far-field and noisy environment adaptations using beamforming or advanced denoising techniques.
- Conducting real-world usability studies to evaluate long-term reliability and user satisfaction.
- Exploring federated learning to personalize the system without compromising user privacy.

The findings from this work contribute to the growing field of TinyML-enabled smart environments, highlighting the potential of edge AI for secure, real-time, and efficient voice interaction systems.

Acknowledgements The authors are thankful to Manipal University for providing support for open-access publishing and providing facilities for conducting this research work.

Author contributions Timothy Malche and Sandeep Budhani have designed the overall framework of the research. Pramod Kumar Soni and Govind Murari Upadhyay have designed the DCNN architectures used in this study, and validated the experimental results. Govind Murari Upadhyay and Timothy Malche have collected data and interpreted results to give conclusive remarks. All authors read and approved the final version of the manuscript. All authors reviewed the manuscript. Timothy Malche and Sandeep Budhani have designed the overall framework of the research. Pramod Kumar Soni and Govind Murari Upadhyay has designed the DCNN architectures used in this study, and validated the experimental results. Govind Murari Upadhyay and Timothy Malche has collected data and interpreted results to give conclusive remarks. All authors read and approved the final version of the manuscript. All authors reviewed the manuscript.

Funding Open access funding provided by Manipal University Jaipur.

Data availability The dataset used in this study can be made available from the corresponding author on reasonable request.

Declarations

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in

the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Pez-Espejo Tan Z-H, Hansen JHL, Jensen J. Deep spoken keyword spotting: an overview. *IEEE Access*. 2022;10:4169–99. <https://doi.org/10.1109/ACCESS.2021.3139508>.
2. Heidari A, Shishehlou H, Darbandi M, Navimipour NJ, Yalcin S. A reliable method for data aggregation on the industrial internet of things using a hybrid optimization algorithm and density correlation degree. *Clust Comput*. 2024;27(6):7521–39.
3. Jabraeil Jamali M, Bahrami B, Heidari A, Allahverdizadeh P, Norouzi F. Towards the internet of things: architectures, security, and applications. Springer; 2019.
4. Zhang Y, Suda N, Lai L, Chandra V. Hello edge: Keyword spotting on microcontrollers. *CoRR*; 2017. abs/1711.07128. [arXiv:1711.07128](https://arxiv.org/abs/1711.07128)
5. Guamán S, Calvopiña A, Orta P, Tapia F, Yoo SG. Device control system for a smart home using voice commands: A practical case. In: Proceedings of the 2018 10th international conference on information management and engineering. ICIME 2018. Association for Computing Machinery, New York, NY, USA; 2018. p. 86–9. <https://doi.org/10.1145/3285957.3285977>.
6. Jose C, Wang J, Strimel GP, Khursheed MO, Mishchenko Y, Kulis B. Latency control for keyword spotting; 2022. arXiv preprint [arXiv:2206.07261](https://arxiv.org/abs/2206.07261).
7. Ahmed S, Shumailov I, Papernot N, Fawaz K. Towards more robust keyword spotting for voice assistants. In: 31st USENIX security symposium (USENIX Security 22); 2022. p. 2655–72.
8. Bushur JIM. Hardware/software co-design for keyword spotting on edge devices; 2023. <https://doi.org/10.25394/PGS.22701319.v1>
9. Bushur J, Chen C. Neural network exploration for keyword spotting on edge devices. *Future Internet*. 2023;15(6):219.
10. Wang J, Li S. Keyword spotting system and evaluation of pruning and quantization methods on low-power edge microcontrollers; 2022. [arxiv:2208.02765](https://arxiv.org/abs/2208.02765)
11. Chen G, Parada C, Heigold G. Small-footprint keyword spotting using deep neural networks. In: 2014 IEEE international conference on acoustics, speech and signal processing (ICASSP); 2014. p. 4087–91. <https://doi.org/10.1109/ICASSP.2014.6854370>
12. Majeed R, Abdullah NA, Ashraf I, Zikria YB, Mushtaq MF, Umer M. An intelligent secure and smart home automation system. *Sci Progr*. 2020;2020(1):4087–91. <https://doi.org/10.1155/2020/4579291>.
13. Wang Z, Li X, Zhou J. Small-footprint keyword spotting using deep neural network and connectionist temporal classifier. *CoRR*; 2017. abs/1709.03665. [arxiv:1709.03665](https://arxiv.org/abs/1709.03665).
14. Choi S, Seo S, Shin B, Byun H, Kersner M, Kim B, Kim D, Ha S. Temporal convolution for real-time keyword spotting on mobile devices. *CoRR*; 2019. abs/1904.03814. [arxiv:1904.03814](https://arxiv.org/abs/1904.03814).
15. Leroy D, Coucke A, Lavril T, Gisselbrecht T, Dureau J. Federated learning for keyword spotting. In: ICASSP 2019—2019 IEEE international conference on acoustics, speech and signal processing (ICASSP); 2019. p. 6341–5. <https://doi.org/10.1109/ICASSP.2019.8683546>
16. Bluche T, Primet M, Gisselbrecht T. Small-footprint open-vocabulary keyword spotting with quantized LSTM networks. *CoRR*; 2020. abs/2002.10851. [arxiv:2002.10851](https://arxiv.org/abs/2002.10851).
17. Hard A, Partridge K, Nguyen C, Subrahmanya N, Shah A, Zhu P, Moreno IL, Mathews R. Training keyword spotting models on non-IID data with federated learning; 2020. [arxiv:2005.10406](https://arxiv.org/abs/2005.10406)
18. Mittermaier S, Kürzinger L, Waschneck B, Rigoll G. Small-footprint keyword spotting on raw audio data with sinc-convolutions. In: ICASSP 2020—2020 IEEE international conference on acoustics, speech and signal processing (ICASSP); 2020. p. 7454–8. <https://doi.org/10.1109/ICASSP40776.2020.9053395>
19. Rybakov O, Kononenko N, Subrahmanya N, Visontai M, Lorenzo S. Streaming keyword spotting on mobile devices. In: Interspeech 2020. ISCA; 2020. <https://doi.org/10.21437/interspeech.2020.1003>
20. Stahl R, Hoffman A, Mueller-Gritschneider D, Gerstlauer A, Schlichtmann U. Deepertings fully distributed CNN inference on resource-constrained edge devices. *Int J Parallel Progr*. 2021;49:600–24. <https://doi.org/10.1007/s10766-021-00712-3>.
21. Espejo LZH, Hansen JHL, Jensen J. Deep spoken keyword spotting: an overview. *IEEE Access*. 2022;10:4169–99. <https://doi.org/10.1109/ACCESS.2021.3139508>.
22. Liu Z, Li T, Zhang P. RNN-T based open-vocabulary keyword spotting in mandarin with multi-level detection. In: ICASSP 2021—2021 IEEE international conference on acoustics, speech and signal processing (ICASSP); 2021. p. 5649–53. <https://doi.org/10.1109/ICASSP39728.2021.9413588>
23. Shrivastava A, Kundu A, Dhir C, Naik D, Tuzel O. Optimize what matters: Training DNN-HMM keyword spotting model using end metric. In: ICASSP 2021—2021 IEEE international conference on acoustics, speech and signal processing (ICASSP); 2021. p. 4000–4. <https://doi.org/10.1109/ICASSP39728.2021.9414797>
24. Riku Immonen TH. Tiny machine learning for resource-constrained microcontrollers. *J Sens*. 2022. <https://doi.org/10.1155/2022/7437023>.
25. Alajlan NN, Ibrahim DM. Tinyml: enabling of inference deep learning models on ultra-low-power IoT edge devices for ai applications. *Micromachines*. 2022. <https://doi.org/10.3390/mi13060851>.
26. He K, Chen D, Su T. A configurable accelerator for keyword spotting based on small-footprint temporal efficient neural network. *Electronics*. 2022. <https://doi.org/10.3390/electronics11162571>.
27. Ng D, Chen Y, Tian B, Fu Q, Chng ES. Convmixer: feature interactive convolution with curriculum learning for small footprint and noisy far-field keyword spotting. In: ICASSP 2022—2022 IEEE international conference on acoustics, speech and signal processing (ICASSP); 2022. p. 3603–7. <https://doi.org/10.1109/ICASSP43922.2022.9747025>

28. Ahmed S, Shumailov I, Papernot N, Fawaz K. Towards more robust keyword spotting for voice assistants. In: 31st USENIX security symposium (USENIX Security 22). USENIX Association, Boston, MA; 2022. p. 2655–72. <https://www.usenix.org/conference/usenixsecurity22/presentation/ahmed>
29. Alzoubi A. Machine learning for intelligent energy consumption in smart homes. *Int J Comput Inf Manuf IJCIM*. 2022. <https://doi.org/10.54489/ijcim.v2i1.75>.
30. Liu B, Wang Z, Zhu W, Sun Y, Shen Z, Huang L, Li Y, Gong Y, Ge W. An ultra-low power always-on keyword spotting accelerator using quantized convolutional neural network and voltage-domain analog switching network-based approximate computing. *IEEE Access*. 2019;7:186456–69. <https://doi.org/10.1109/ACCESS.2019.2960948>.
31. Park DS, Chan W, Zhang Y, Chiu C-C, Zoph B, Cubuk ED, Le QV. SpecAugment: a simple data augmentation method for automatic speech recognition. In: *Interspeech*; 2019. <https://doi.org/10.21437/interspeech.2019-2680>
32. Mushtaq Z, Su SF. Environmental sound classification using a regularized deep convolutional neural network with data augmentation. *Appl Acoust*. 2020;167: 107389. <https://doi.org/10.1016/j.apacoust.2020.107389>.
33. Shan W, Yang M, Wang T, Lu Y, Cai H, Zhu L, Xu J, Wu C, Shi L, Yang J. A 510-nw wake-up keyword-spotting chip using serial-FFT-based MFCC and binarized depthwise separable CNN in 28-nm cmos. *IEEE J Solid-State Circ*. 2021;56(1):151–64. <https://doi.org/10.1109/JSSC.2020.3029097>.
34. Deng M, Meng T, Cao J, Wang S, Zhang J, Fan H. Heart sound classification based on improved MFCC features and convolutional recurrent neural networks. *Neural Netw*. 2020;130:22–32. <https://doi.org/10.1016/j.neunet.2020.06.015>.
35. Xiang L, Lu S, Wang X, Liu H, Pang W, Yu H. Implementation of LSTM accelerator for speech keywords recognition. In: 2019 IEEE 4th international conference on integrated circuits and microsystems (ICICM). 2019. p. 195–198. <https://doi.org/10.1109/ICICM48536.2019.8977176>
36. Kaur K, Jain N. Feature extraction and classification for automatic speaker recognition system—a review. *Int J Adv Res Comput Sci Softw Eng*. 2015;5:1–6.
37. Lei J, Rahman T, Shafik R, Wheeldon A, Yakovlev A, Granmo O-C, Kawsar F, Mathur A. Low-power audio keyword spotting using Tsetlin machines. *J Low Power Electr Appl*. 2021. <https://doi.org/10.3390/jlpea11020018>.
38. Audio MFCC. <https://docs.edgeimpulse.com/docs/edge-impulse-studio/processing-blocks/audio-mfcc>. Accessed 1 Mar 2024.
39. Arik SO, Kliegl M, Child R, Hestness J, Gibiansky A, Fougner C, Prenger R, Coates A. Convolutional recurrent neural networks for small-footprint keyword spotting; 2017. arXiv preprint [arXiv:1703.05390](https://arxiv.org/abs/1703.05390).
40. Yoon J, Kim N, Lee D, Lee S-J, Kwak G-H, Kim T-H. A resource-efficient keyword spotting system based on a one-dimensional binary convolutional neural network. *Electronics*. 2023;12(18):3964.
41. Daniel S Park WC. SpecAugment: a new data augmentation method for automatic speech recognition; 2019. <https://research.google/blog/specaugment-a-new-data-augmentation-method-for-automatic-speech-recognition/>. Accessed 18 Mar 2025.
42. Suda N, Chandra V, Dasika G, Mohanty A, Ma Y, Vrudhula S, Seo J-S, Cao Y. Throughput-optimized opencl-based FPGA accelerator for large-scale convolutional neural networks. In: *Proceedings of the 2016 ACM/SIGDA international symposium on field-programmable gate arrays. FPGA'16*. Association for computing machinery, New York, NY, USA; 2016. p. 16–25. <https://doi.org/10.1145/2847263.2847276>
43. Qiu J, Wang J, Yao S, Guo K, Li B, Zhou E, Yu J, Tang T, Xu N, Song S, Wang Y, Yang H. Going deeper with embedded fpga platform for convolutional neural network. In: *Proceedings of the 2016 ACM/SIGDA international symposium on field-programmable gate arrays. FPGA'16*. Association for computing machinery, New York, NY, USA; 2016. p. 26–35. <https://doi.org/10.1145/2847263.2847265>
44. Lai L, Suda N, Chandra V. Deep convolutional neural network inference with floating-point weights and fixed-point activations; 2017. [arxiv:1703.03073](https://arxiv.org/abs/1703.03073)
45. Arduino Nano 33 BLE sense overview. <https://gilberttanner.com/blog/arduino-nano-33-ble-sense-overview/>. Accessed 20 Apr 2025.
46. Javed A, Malik KM, Irtaza A, Malik H. Towards protecting cyber-physical and IoT systems from single- and multi-order voice spoofing attacks. *Appl Acoust*. 2021;183: 108283. <https://doi.org/10.1016/j.apacoust.2021.108283>.
47. Ayaz F, Zakariyya I, Cano J, Keoh SL, Singer J, Pau D, Kharbouche-Harrari M. Improving robustness against adversarial attacks with deeply quantized neural networks; 2023. [arxiv:2304.12829](https://arxiv.org/abs/2304.12829)